

PAN Card Verification & AML Monitoring System

Comprehensive Technical Documentation

Generated: April 07, 2026

PROJECT OVERVIEW

This is an end-to-end **KYC (Know Your Customer) and AML (Anti-Money Laundering) verification system** for Indian PAN card users. It combines AI/ML algorithms across three major modules:

Key Components

- Module 1: Document Verification - Detects forged/tampered PAN cards using 6 authenticity checks
- Module 2: Biometric Authentication - Face recognition + liveness detection + behavioral biometrics
- Module 3: AML Monitoring - Rule-based + ML anomaly detection for transaction monitoring

MODULE 1: DOCUMENT VERIFICATION (Forgery Detection)

Purpose

Verify that uploaded PAN card is authentic (not forged/manipulated) and readable. Uses OCR extraction plus 6 parallel image analysis checks.

OCR Extraction Process

- Image preprocessing: denoise, CLAHE contrast enhancement, Otsu binarization
- Text extraction via EasyOCR (primary) or Tesseract (fallback)
- Field extraction: PAN number, name, date of birth via regex patterns
- Validation scoring: 1.0 (perfect), 0.5 (partial), 0.0 (missing)

Six Authenticity Checks

1. **Font Consistency (15%):** Detects mixed fonts from different sources
2. **Edge Consistency (20%):** Detects copy-paste, clone stamping, edited regions
3. **Color Histogram (15%):** Detects color inconsistencies from pasting
4. **Structural Alignment (10%):** Verifies PAN card layout integrity
5. **Metadata Analysis (5%):** Checks EXIF for editing software traces
6. **SSIM Score (5%):** Structural similarity for internal consistency

Document Score Verdict:
>= 0.75 = AUTHENTIC | 0.50-0.74 = SUSPICIOUS | < 0.50 = LIKELY FORGED

MODULE 2: BIOMETRIC AUTHENTICATION

Sub-Components

- Face Matching:** Compares PAN card photo with selfie using FaceNet embeddings
- Liveness Detection:** Detects spoof attacks (printed photos, screens, 3D masks)
- Behavioral Biometrics:** Analyzes keystroke dynamics and mouse movement patterns

Face Matching Details

Extract faces from PAN and selfie. Generate 128-dimensional embeddings using DeepFace/FaceNet. Compute Euclidean distance. Convert to similarity score (0-1). Verdict: distance < threshold = MATCH.

Liveness Detection Methods

- LBP Texture Analysis (35%): Real skin has natural texture; prints have patterns
- FFT Frequency Analysis (25%): Detects moiré patterns and periodic artifacts
- Gradient Statistics (25%): Real faces have smooth gradients; prints are quantized
- Specular Highlights (15%): Real faces have natural reflections; prints are flat

Liveness Verdict:
>= 0.65 = LIVE | 0.45-0.64 = UNCERTAIN | < 0.45 = SPOOF DETECTED

MODULE 3: AML TRANSACTION MONITORING

Three Sub-Engines

Rule-Based Engine (40% weight): 10 FATF/RBI compliance rules (large transactions, structuring, high-risk countries, etc.)

Isolation Forest (35% weight): Statistical anomaly detection on 15-dimensional transaction features

LSTM Sequential Monitor (25% weight): Detects AML patterns: layering, smurfing, rapid fund movement

Rule Engine Examples

- R1: Transaction $\geq$ INR 50,000 (CTR threshold)
- R2: Multiple transactions $< 50K$ totaling $\geq 50K$ in 24h (structuring)
- R4: Transfer to high-risk country (FATF blacklist)
- R7: Funds credited then debited within 2h (rapid movement)
- R10: Dormant account (180+ days) suddenly active

AML Score Verdict:

≥ 0.70 = CLEAN | $0.40-0.69$ = MONITOR | < 0.40 = SUSPICIOUS

FRONTEND INTEGRATION SUMMARY

Upload Endpoint

```
POST /verify/upload/
```

```
Input: full_name, pan_card file, selfie_image file
```

```
Output: doc_score, biometric_score, aml_score, final_status
```

Display Components Needed

- Document verification: Score percentage, 6-feature breakdown table, verdict (authentic/suspicious/forged)
- Biometric results: Face match score, liveness score, behavioral score, combined verdict
- AML results: AML score, triggered rules with severity (HIGH/MEDIUM/LOW), recommendations
- Final decision: APPROVED / MANUAL_REVIEW / REJECTED with explanation

Tech Stack

- Backend: Django 4.2+, Django REST Framework
- CV/ML: OpenCV, scikit-learn, DeepFace, EasyOCR
- Database: Django ORM (SQLite/PostgreSQL)
- Deployment: Docker recommended

Document Generated: April 07, 2026 at 15:08
For detailed implementation, see PROJECT_SUMMARY.md